

ENTRADA Y SALIDA ESTÁNDAR

PROFESOR: Federico Melo Barrero

CÓMO FUNCIONAN LA ENTRADA Y LA SALIDA ESTÁNDAR

En todos los problemas del curso, las soluciones se entregan como un programa que lee desde la entrada estándar (*standard input*) y escribe en la salida estándar (*standard output*). No se reciben argumentos por línea de comandos ni se abren archivos desde el código: el programa siempre lee y escribe usando los mecanismos estándar del lenguaje elegido (por ejemplo, `sys.stdin` en Python, `System.in` en Java, `cin` en C++, o `readline` en JavaScript).

FORMATO DE LOS CASOS DE PRUEBA

Todo problema del curso comparte la misma estructura general de entrada: la primera línea contiene un entero T , la cantidad de casos de prueba. A continuación siguen T bloques, uno por caso de prueba, cuyo formato específico define cada problema. La salida sigue la misma lógica: se produce una respuesta por cada uno de los T casos, en el mismo orden en que se leyeron.

REDIRECCIÓN Y EJECUCIÓN

Para probar un programa sin escribir cada caso a mano, la consola permite redirigir un archivo hacia la entrada estándar y capturar la salida estándar en otro archivo, sin modificar una sola línea del programa: `<` redirige el contenido de un archivo hacia la entrada estándar, y `>` redirige la salida estándar hacia un archivo.

Suponiendo una solución en un archivo llamado `Problema`, con los casos de prueba en `entrada.in` y la salida esperada en `salida.out`, el comando depende del lenguaje elegido. Solo hace falta el bloque correspondiente al lenguaje de cada estudiante.

Java (requiere compilar antes con `javac Problema.java`):

```
1 java Problema < entrada.in > salida.out
```

Python:

```
1 python Problema.py < entrada.in > salida.out
```

C++ (requiere compilar antes, e.g. `g++ Problema.cpp -o Problema`):

```
1 ./Problema < entrada.in > salida.out
```

JavaScript:

```
1 node Problema.js < entrada.in > salida.out
```

EJEMPLO: PROBLEMA P0

El Problema P0 es un problema de juguete que ilustra, con un enunciado sencillo, cómo se espera que se lean los datos de entrada y se escriban las respuestas.

ENUNCIADO

Entrada: La primera línea contiene la cantidad de casos de prueba T . Cada una de las siguientes T líneas contiene una lista de n números enteros ($0 < n \leq 1000$), separados por espacio.

Salida: Por cada caso de prueba se espera una línea de la forma $c_p \ sp$, donde c_p es la cantidad de enteros pares leídos y sp es la suma de esos enteros pares.

CASO DE EJEMPLO

```
1 Entrada
2 6
3 1 2 3 4 5 6 7 8 9 10
4 1 -2 3 -4 5 -6 7 -8 9 -10
5 2 4 6 8
6 1 3 5 7
7 1 1 1 1 1
8 -2 2 -2 2 -2 2
9
10 Salida
11 5 30
12 5 -30
13 4 20
14 0 0
15 0 0
16 6 0
```

EJECUCIÓN

Con los casos de prueba anteriores guardados en `P0.in`:

Java:

```
1 java ProblemaP0 < P0.in > P0.out
```

Python:

```
1 python ProblemaP0.py < P0.in > P0.out
```

C++:

```
1 ./ProblemaP0 < P0.in > P0.out
```

JavaScript:

```
1 node ProblemaP0.js < P0.in > P0.out
```